Exercise 09: PWM

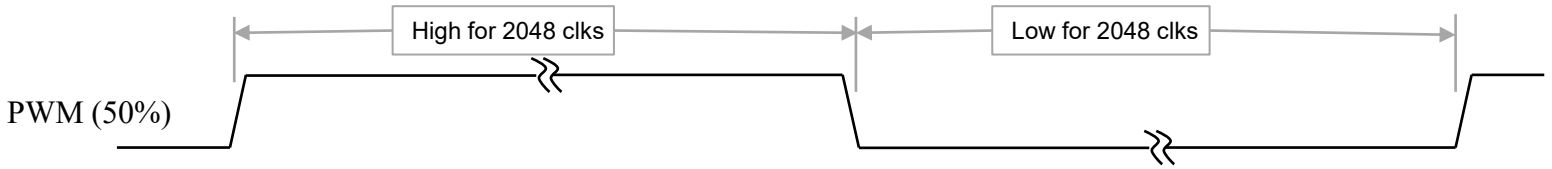
- We have to have a way of varying the strength/direction of the drive to the MazeRunner's motors
  → This will be done through **P**ulse **W**idth **M**odulation (PWM)

- PWM is commonly used as a simple way of varying intensity
  - The signal typically will have a fixed frequency and the duty cycle will determine the magnitude.
  - E.g., turn the LED on at full brightness for 100usec then off for 100usec. The human eye will average the light intensity (your retina integrates), so the light will look like the LED is driven at ½ intensity.

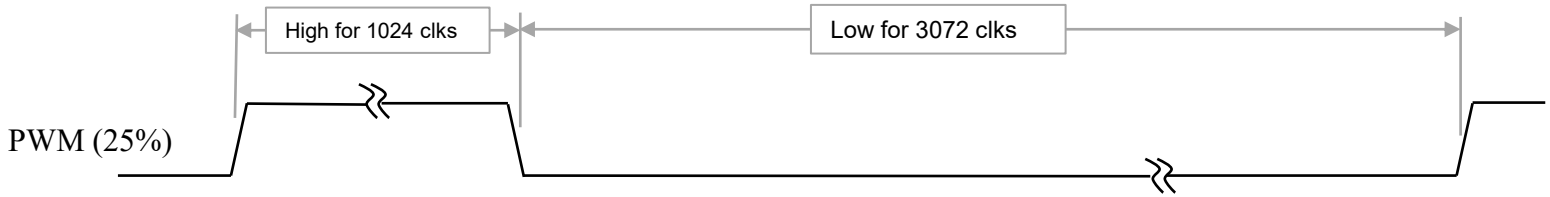
- The same works with motor control. Drive the motor coil at full voltage for 50usec and off for 150usec. The inductance in the coil will "average" the current and it will look like the motor is driven at 25%.

Exercise 09: PWM

- Consider a **12-bit** PWM signal being driven at 50% duty cycle
- The period of the PWM waveform is 4096 clocks (2^{12})
  → Since our system clock is 50MHz this is 80usec



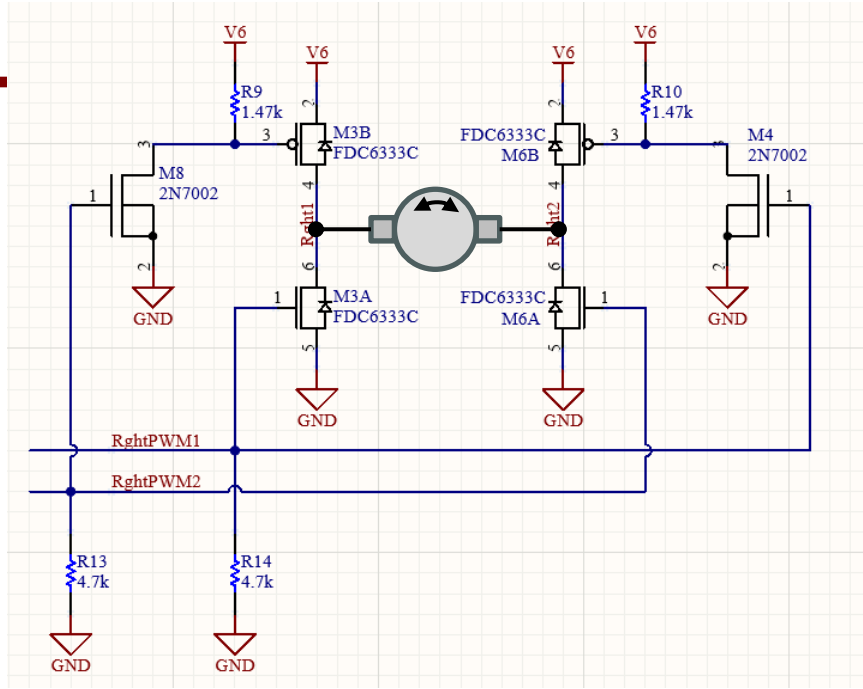
- At 25% duty cycle:

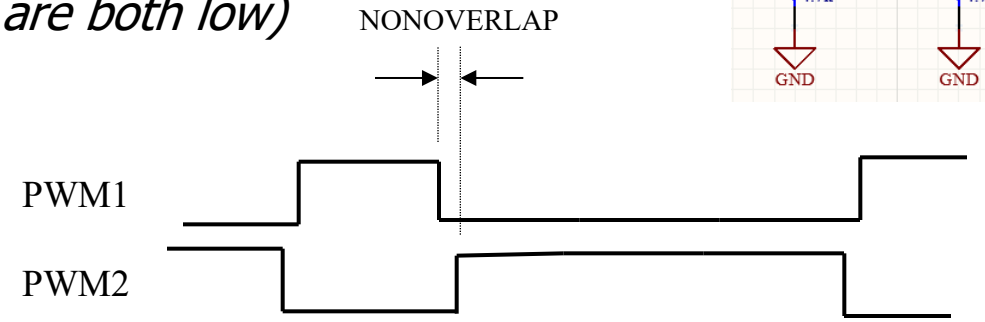


- Note: We cannot achieve both 0 duty cycle and 100% duty cycle
  - Think about it… If 0 means zero duty then what does 0xFFF mean? It means we are on for 4095 out of 4096 clocks, so not quite 100%. But we are fine with this.

Exercise 09: PWM

- The motors can be driven forward or reverse from the 6V battery through the circuit shown.
- PWM1 & PWM2 are **complementary** signals *(opposite of each other)* but they also have a **non-overlap time** *(time in which they are both low)*



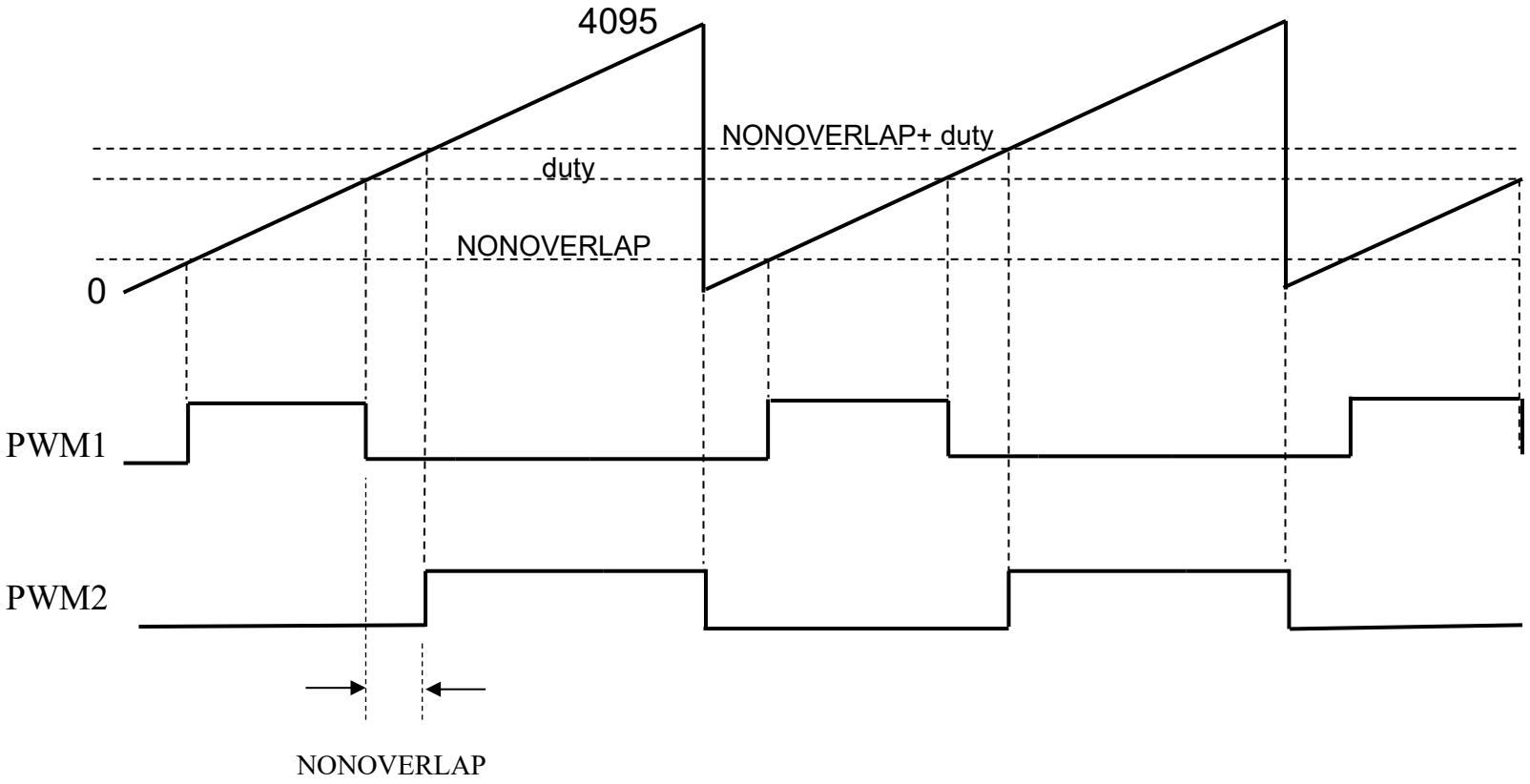


Why is the non-overlap necessary?

- If both the NMOS and PMOS power transistors were on, then current would be shooting through the transistors from 6V supply to ground.
- We need to ensure both NMOS and PMOS in a stack are never on at the same time.

Exercise 09: PWM Implementation

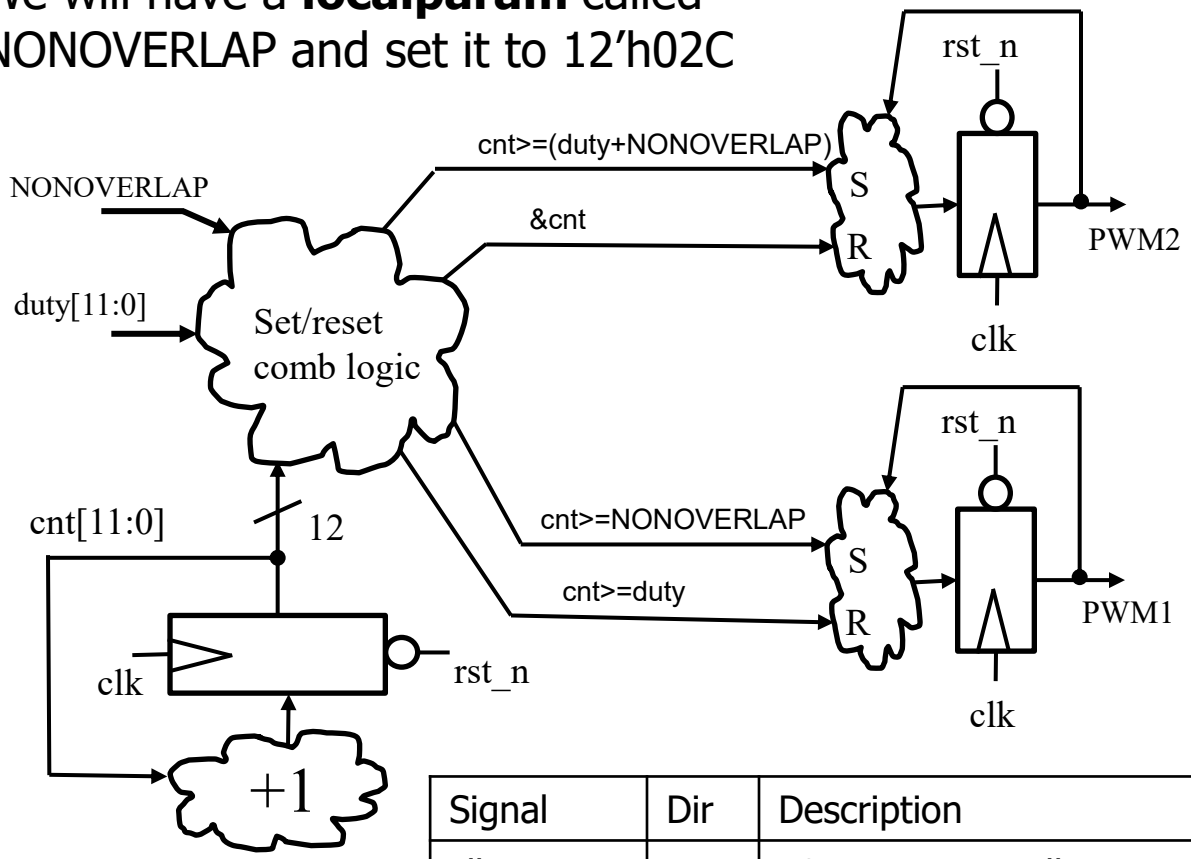
- Think of a 12-bit free-running counter



- PWM1 runs from **NONOVERLAP** to **duty**
- PWM2 runs from **duty+NONOVERLAP** to 2047
- Duty cycles less than NONOVERLAP will result in zero duty cycle

Exercise 09: PWM Implementation

We will have a **localparam** called NONOVERLAP and set it to 12'h02C



Having PWM signals direct from flops ensures no glitching.

(You already made an S/R flop of this style in HW2.)

Signal	Dir	Description
clk	in	50MHz system clk
rst_n	in	Asynch active low
duty[11:0]	in	Specifies duty cycle (unsigned 12-bit)
PWM1 PWM2	out	Complementary glitch free PWM signals with non-overlap

Exercise 09: PWM

- Implement **PWM12.sv**

- Implement **PWM12_tb.sv** & generate waveforms showing various duty cycles
  - Probably want to test at a low duty cycle, a mid duty cycle, and a high duty cycle
  - Self-checking not necessary